



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения (ИиППО)

ЗАДАНИЕ на выполнение курсовой работы

по дисциплине: «Фронтенд-разработка»

по профилю: Разработка программных продуктов и проектирование информационных систем
направления профессиональной подготовки: Программная инженерия (09.03.04)

Студент: Романов Н.С.

Группа: ИКБО-11-23

Срок представления к защите: 29.05.2025

Руководитель: _____

Тема Приложение для составления кулинарных рецептов

Исходные данные:используемые технологии: HTML5, CSS3, JS TS, React, нормативный документ: инструкция по организации и проведению курсового проектирования СМК МИРЭА 7.5.1/04.И.05-18, ГОСТ 7.32-2017.

Перечень вопросов, подлежащих разработке, и обязательного графического материала:

1. Провести анализ предметной области разрабатываемого веб-сайта. 2. Сформировать требования к разрабатываемому веб-сайту. 3. Сформировать концепцию веб-сайта. 4. Обосновать выбор технологий разработки веб-сайта. 5. Разработать дизайн интерфейса веб-сайта 6. Реализовать фронтенд веб-сайта. 7. Разработка теста для проверки функционала веб-сайта 8. Создать презентацию по выполненной курсовой работе.

Руководителем произведён инструктаж по технике безопасности, противопожарной технике и правилам внутреннего распорядка.

Зав. кафедрой ИиППО: [подпись] / Болбаков Р. Г. /, « 13 » 03 2025 г.

Задание на КР выдал: [подпись] / Болбаков Р. Г. /, « 13 » 03 2025 г.

Задание на КР получил: [подпись] Романов Н.С., « 13 » 03 2025 г.

Оглавление

Введение.....	4
Глава 1. Теоретическая часть (предметная область).....	6
1.1 Анализ конкурентных решений.....	6
1.2 Формирование требований к продукту.....	12
Глава 2. Концепция продукта и техническая реализация	13
2.1. Концепция продукта	13
2.2. Обоснование принимаемых технических решений.....	13
2.3. Обоснование выбора инструментария и компонентов	15
2.4. Дизайн интерфейса	16
2.5 Дизайн в Figma	17
2.6. Заключение по главе	18
Глава 3. Реализация.....	19
3.1. Реализация.....	19
3.2 Основные модули	19
3.3 Заключение по главе	26
Глава 4. Тестирование.....	27
Заключение	37
Список использованных источников	39

Введение

Современные технологии оказывают значительное влияние на все сферы жизни человека, включая быт и питание. В условиях стремительного ритма жизни люди всё чаще обращаются к цифровым решениям, которые позволяют оптимизировать повседневные задачи, в том числе приготовление пищи. Одним из актуальных направлений в области цифровых продуктов являются веб-приложения, помогающие пользователям находить рецепты, основываясь на доступных ингредиентах. Такие сервисы не только экономят время, но и упрощают процесс принятия решений, связанных с выбором блюд.

Целью данной курсовой работы является разработка веб-приложения, которое позволяет пользователю выбирать категорию блюда и имеющиеся ингредиенты, а затем предоставляет список подходящих рецептов. Это решение направлено на создание удобного инструмента для поиска рецептов, который может быть использован в повседневной жизни.

Для достижения поставленной цели необходимо решить следующие задачи:

- Анализ предметной области: изучение современных тенденций в сфере цифровых продуктов, связанных с кулинарией, а также выявление ключевых потребностей целевой аудитории.
- Анализ конкурентных решений: исследование существующих аналогичных сервисов, их функциональности, преимуществ и недостатков.
- Формирование требований: определение технических и функциональных требований к будущему веб-приложению, исходя из потребностей пользователей и особенностей предметной области.
- Создание концепции продукта: разработка общей структуры и логики работы приложения.
- Выбор инструментов: обоснование выбора технологий, языков программирования и других инструментов для реализации проекта.

- Создание дизайна интерфейса: проектирование удобного и интуитивно понятного пользовательского интерфейса.
- Программирование: реализация всех компонентов приложения, обеспечивающих выполнение необходимой функциональности.
- Тестирование: проверка работоспособности, надежности и соответствия требованиям созданного приложения.

Разработанное в рамках курсовой работы веб-приложение должно стать примером эффективного использования современных технологий для решения повседневных задач, а также продемонстрировать навыки анализа, проектирования и реализации программных продуктов.

Глава 1. Теоретическая часть (предметная область)

Разработка веб-приложений, направленных на упрощение процесса выбора и приготовления блюд, является актуальной задачей в сфере цифровых технологий. Такие сервисы позволяют пользователям находить рецепты, основываясь на доступных ингредиентах, предпочтениях по категории блюда и других параметрах. Это особенно важно в условиях современной жизни, когда экономия времени и минимизация усилий становятся ключевыми факторами.

Сайты и мобильные приложения для подбора рецептов обычно используют базу данных с рецептами, фильтруемыми по различным критериям: наличие ингредиентов, тип блюда, уровень сложности, время приготовления и т. д. Пользователь может выбрать продукты, которые у него есть, и система подберет подходящие рецепты. В некоторых случаях предусмотрена возможность сохранять избранные рецепты, просматривать детали приготовления, делиться результатами и оставлять отзывы.

Такой подход позволяет не только оптимизировать процесс принятия решений, но и способствует более рациональному использованию продуктов, что снижает риск их перерасхода или порчи.

1.1 Анализ конкурентных решений

В интернете существует множество сайтов и приложений, предоставляющих функционал поиска рецептов:

1. AllRecipes — один из самых популярных сайтов, где пользователи могут искать рецепты по ингредиентам, категории, уровню сложности. Предоставляет рейтинг, комментарии, фото от пользователей.

Достоинства:

- Огромная база рецептов — один из самых популярных сайтов в мире, с десятками тысяч рецептов на любой вкус.

- Фильтры поиска: можно искать по ингредиентам, уровню сложности, времени приготовления, кухне, диете (например, без глютена, вегетарианские).
- Рейтинги и отзывы: пользователи оценивают рецепты и делятся личным опытом, что помогает выбрать лучшие варианты.
- Фотографии блюд от пользователей: наглядное представление результата.
- Мобильное приложение: удобно использовать в процессе готовки.
- Сообщество: активное сообщество пользователей, обмен мнениями, советами.

Недостатки:

- Реклама: в бесплатной версии много рекламы, что может мешать чтению.
- Регистрация для некоторых функций: например, сохранение рецептов в «избранное» возможно только после регистрации.
- Иногда слишком много контента: сложно найти действительно уникальные или малоизвестные рецепты среди массы стандартных.
- Много англоязычных рецептов, что может быть неудобно для носителей других языков.

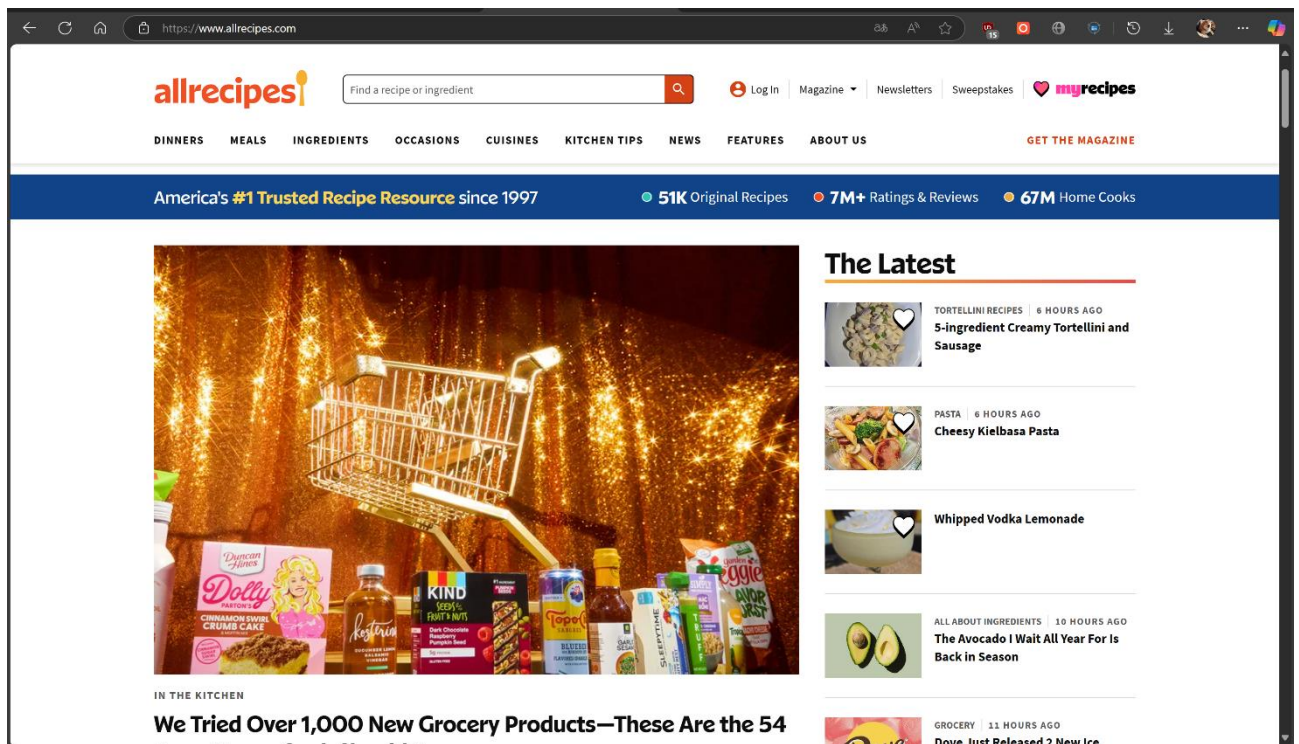


Рисунок 1 – Интерфейс Allrecipes

2. Yummly — предлагает поиск по ингредиентам и диетическим ограничениям. Также имеет функцию генерации списка покупок.

Достоинства:

- Персонализация: сайт учитывает ваши предпочтения, аллергии, диеты и предлагает рецепты под них.
- Поиск по ингредиентам и ограничениям: удобен для людей с особыми требованиями к питанию.
- Автоматическое формирование списка покупок: полезная функция для планирования.
- Приложение с хорошим UX: интуитивно понятный интерфейс.
- Шаговые инструкции: подробные этапы приготовления.

Недостатки:

- Платная подписка (Yummly Pro): расширенный функционал доступен только за деньги.

- Небольшая база рецептов по сравнению с AllRecipes.
- Ограниченная социальная активность: нет полноценного взаимодействия между пользователями.
- Мало фото от пользователей, больше профессиональных изображений.

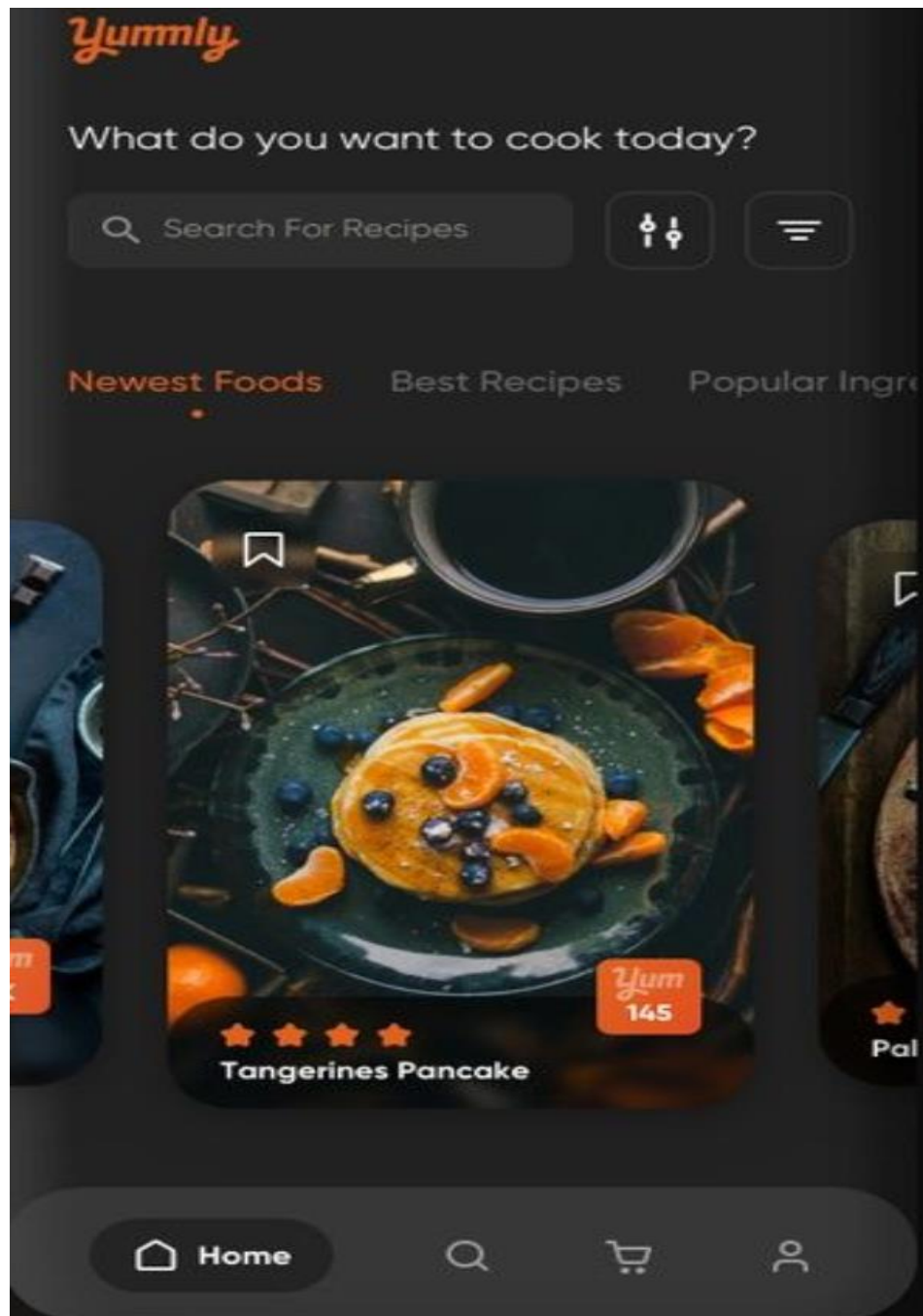


Рисунок 2 – Интерфейс Yummly

3. Едим Дома — русскоязычный сайт с большим количеством рецептов, возможностью поиска по продуктам и категориям.

Достоинства:

- Русскоязычная аудитория: удобен для пользователей из СНГ.
- Большое количество рецептов от простых до сложных.
- Простой и понятный интерфейс: ориентирован на русскоязычного пользователя.
- Поиск по продуктам: можно находить рецепты, исходя из того, что есть в холодильнике.
- Видео-рецепты: дополнительный способ обучения.
- Поддержка различных категорий и фильтров.

Недостатки:

- Устаревший дизайн: выглядит менее современно по сравнению с зарубежными аналогами.
- Мало мультимедийного контента: не все рецепты имеют фотографии или видео.
- Ограниченная персонализация: нет глубоких настроек под диеты, аллергии и т.п.
- Реклама: также присутствует, иногда может мешать.

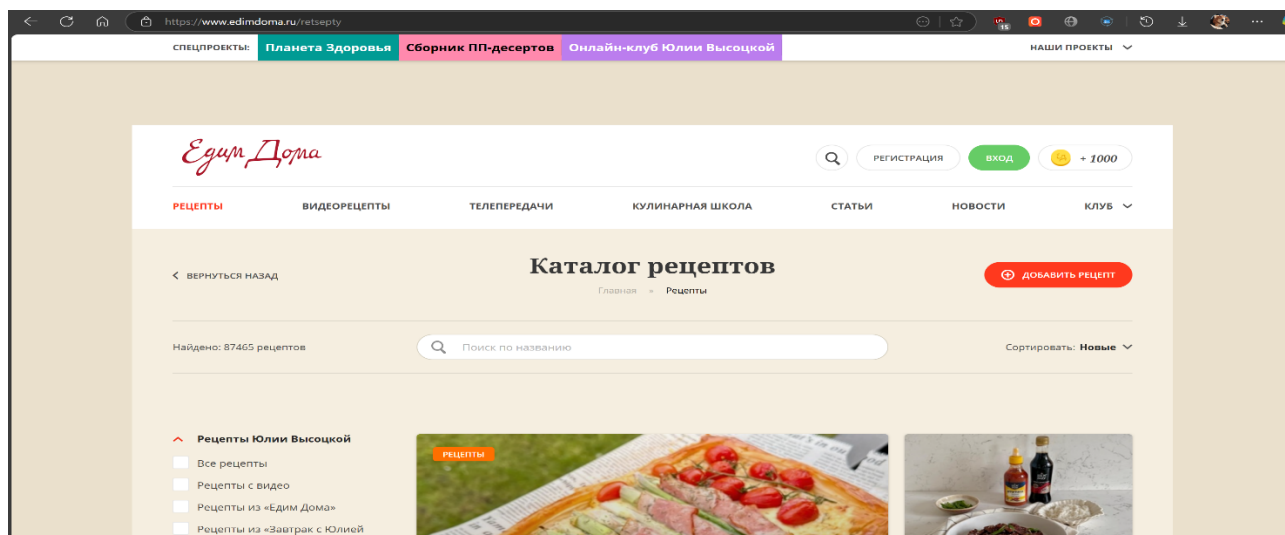


Рисунок 3 – Интерфейс Едим Дома

4. Cookpad — социальная платформа, где пользователи делятся своими рецептами, а также могут искать блюда по наличию продуктов.



Рисунок 4 – Интерфейс Cookpad

Достоинства:

- Социальный элемент: пользователи могут делиться рецептами, комментировать, задавать вопросы.
- Поиск по наличию продуктов: одна из ключевых особенностей — вводишь, что есть дома, получаешь подходящие рецепты.
- Мировые кухни: большое разнообразие рецептов из разных стран.
- Мобильное приложение: удобно использовать на кухне.

- Бесплатный доступ ко всему функционалу: большинство возможностей доступны без платы.

Недостатки:

- Качество рецептов различается: так как всё добавляется пользователями, встречаются некачественные или плохо оформленные рецепты.
- Отсутствие модерации: некоторые рецепты могут быть небезопасными или содержать ошибки.
- Много повторов: одни и те же рецепты могут быть опубликованы несколькими пользователями.
- Ограниченные фильтры по диетическим ограничениям.

1.2 Формирование требований к продукту

На основе анализа предметной области и существующих решений были сформированы следующие требования к разрабатываемому веб-приложению:

Функциональные требования:

- Выбор категории блюда (закуска, суп, напиток, основное блюдо и др.);
- Выбор ингредиентов из предложенного списка;
- Отображение списка подходящих рецептов с возможностью просмотра деталей;
- Возможность добавления рецепта в избранное;

Нефункциональные требования:

- Простота и интуитивность интерфейса;
- Быстродействие;
- Совместимость с популярными браузерами;
- Легкость дальнейшего расширения и модификации.

Глава 2. Концепция продукта и техническая реализация

2.1. Концепция продукта

Целью данной курсовой работы являлась разработка веб-приложения, позволяющего пользователю выбирать категорию блюда и указывать имеющиеся ингредиенты с последующим выводом подходящих рецептов. Такой подход к организации поиска рецептов делает приложение удобным для использования как опытными кулинарами, так и новичками.

Концепция проекта предполагала создание простого, но функционального интерфейса, состоящего из следующих этапов:

1. Главная страница — приветствие и кнопка «Начать».
2. Выбор категории блюда — пользователь выбирает тип блюда (например: закуска, суп, основное блюдо).
3. Выбор ингредиентов — несколько страниц, на которых пользователь указывает доступные ему продукты, разделенные по группам (мясо, овощи, животные продукты и т. д.).
4. Вывод списка рецептов — отображение подходящих рецептов с возможностью просмотра подробностей и добавления в избранное.
5. Избранное — сохранённые рецепты.
6. О наших рецептах — описание философии и преимуществ используемых рецептов.

Для обеспечения удобства навигации был реализован единый стиль оформления, логическая структура меню.

2.2. Обоснование принимаемых технических решений

При выборе технологий и архитектуры приложения учитывались следующие факторы:

- Простота разработки и поддержки;
- Возможность масштабирования;
- Поддержка клиентской логики без серверной части (для демонстрационных целей);

На основе этих требований был выбран следующий стек технологий:

Используемые технологии:

- React — популярная JavaScript-библиотека для создания пользовательских интерфейсов. Позволяет разрабатывать компонентную архитектуру, что облегчает тестирование, повторное использование кода и дальнейшее развитие приложения.
- TypeScript — обеспечивает строгую типизацию, повышает читаемость и надёжность кода.
- React Router — используется для маршрутизации между страницами.
- React Context API — позволяет централизованно управлять состоянием приложения без необходимости внедрения Redux.
- LocalStorage — для хранения данных пользователя (выбранные категории, ингредиенты, избранное) без серверной части.
- CSS Modules / BEM — для стилизации компонентов с минимальным риском конфликтов классов.

Преимущества выбранных решений:

- Разделение на компоненты позволяет легко модифицировать и расширять функционал.
- Управление состоянием через контекст обеспечивает простоту передачи данных между компонентами.

- LocalStorage предоставляет временное решение для хранения данных без необходимости использования базы данных или сервера.

2.3. Обоснование выбора инструментария и компонентов

Проект был реализован с использованием следующих ключевых компонентов:

1. RecipeContext

Контекст (RecipeContext) используется для управления состоянием приложения:

- Выбранная категория блюда (selectedCategory);
- Выбранные ингредиенты (selectedProducts);
- Список избранных рецептов (favoriteRecipes);
- Логика фильтрации рецептов (filteredRecipes).

Это позволило отказаться от глубокой передачи props и сделать приложение более гибким и управляемым.

2. CategoryGrid

Компонент CategoryGrid представляет собой сетку категорий, где пользователь может выбрать тип блюда. При выборе категории она сохраняется в состоянии и в localStorage.

3. ProductGrid

ProductGrid — универсальный компонент, который используется на нескольких страницах для выбора продуктов. Он принимает список продуктов, предыдущую и следующую страницы. Реализует кликабельные карточки, которые можно выбирать и отменять выбор.

4. RecipeList

Этот компонент отображает список найденных рецептов. Выводится после выбора категории и ингредиентов. Рецепты фильтруются по совпадению

ингредиентов и категории. Также реализована возможность добавления рецепта в избранное.

5. RecipeModal

Модальное окно для просмотра деталей рецепта. Содержит название, полный текст рецепта и возможность добавить/удалить из избранного.

6. Страницы

Все страницы (HomePage, CategoriesPage, MeatPage, VegetablesPage, AnimalOriginPage, PlantOriginPage, RecipeListPage, FavoritesPage, AboutPage) реализованы как отдельные компоненты, подключаемые через маршруты React Router.

2.4. Дизайн интерфейса

Интерфейс приложения был разработан с учетом принципов юзабилити, эргономики и визуальной привлекательности. Основные элементы дизайна:

Цветовая палитра

Основные цвета: теплые тона, напоминающие кухню, и еду (бежевый, коричневый, белый).

Шрифты

- Используются стандартные веб-шрифты: 'Roboto', 'Open Sans', 'Lato'.
- Заголовки — жирные, с увеличенным межстрочным интервалом.
- Текст — легко читаемый, с четкими границами.

Элементы интерфейса

- Модальные окна для просмотра рецептов;
- Изображения фоновых картинок для эмоционального восприятия;
- Иконки для дополнительной визуализации информации.

Навигация

- Верхнее горизонтальное меню с логотипом и ссылками на основные страницы (Главная, О наших рецептах, Избранное).
- Стрелки навигации на страницах выбора ингредиентов и рецептов.
- Активные элементы выделены цветом и стилем.

2.5 Дизайн в Figma

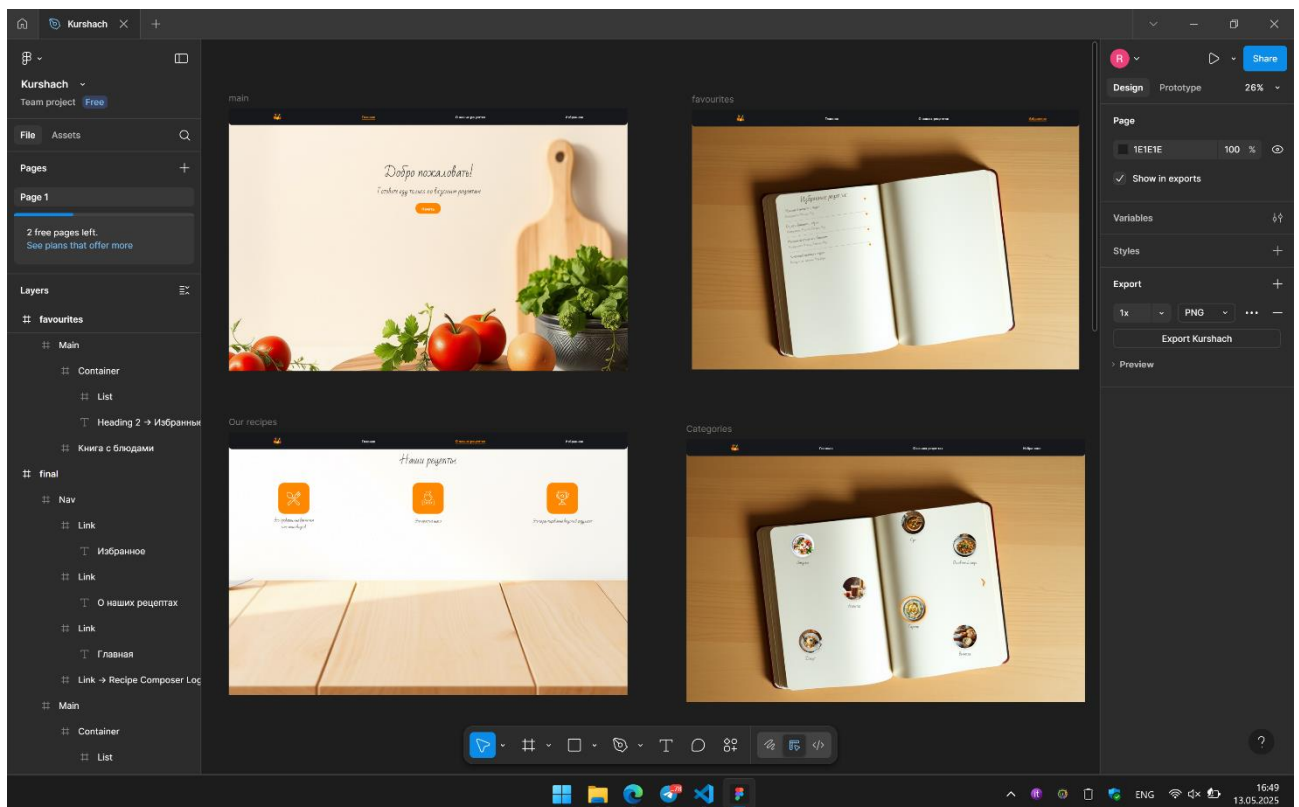


Рисунок 5 – Дизайн основных страниц

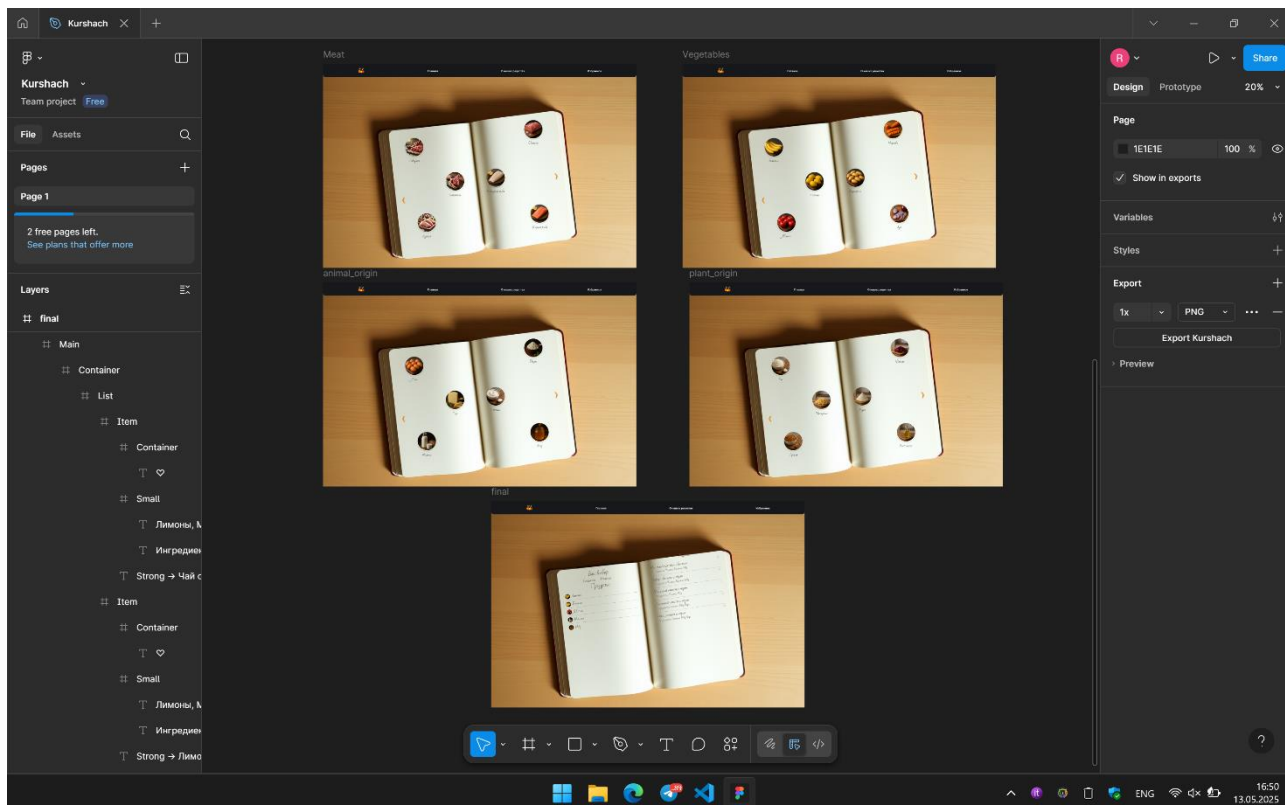


Рисунок 6 – Дизайн страниц выбора и результата

2.6. Заключение по главе

Во второй главе были рассмотрены концепция и техническая реализация веб-приложения. Было обосновано использование современного стека технологий, описаны основные компоненты и их назначение, представлен дизайн интерфейса. Предложенный подход позволил создать функциональное и удобное приложение, которое соответствует всем поставленным требованиям и может быть легко расширено в будущем.

Глава 3. Реализация

3.1. Реализация

В ходе выполнения курсовой работы было реализовано веб-приложение, позволяющее пользователю выбирать категорию блюда и имеющиеся ингредиенты, а затем получать список подходящих рецептов. Приложение написано с использованием современных технологий: React + TypeScript, что обеспечило модульность, читаемость кода и поддержку типизации.

Архитектура проекта

Проект построен на компонентной архитектуре React, где каждый элемент интерфейса представлен отдельным компонентом. Это позволило:

- Разделить логику и представление;
- Повторно использовать компоненты;
- Упростить процесс тестирования и поддержки.

Основные файлы проекта:

- `RecipeContext.tsx` — централизованное управление состоянием приложения (выбранная категория, выбранные продукты, избранное);
- `App.tsx` — маршрутизация страниц с помощью `react-router-dom`;
- Компоненты (`CategoryGrid`, `ProductGrid`, `RecipeList`, `RecipeModal`) — реализуют основную функциональность;
- Данные (`recipes.ts` и `products.ts`) — статические данные о рецептах и ингредиентах.

3.2 Основные модули

RecipeContext

Разработан контекст (`RecipeContext`), управляющий состоянием приложения:

- Хранение выбранной категории (selectedCategory);

```
const [selectedCategory, setSelectedCategory] = useState<string>('');
```

Рисунок 7 – Реализация работы хранения выбранной категории

- Временное хранение выбранных продуктов (selectedProducts);

```
const [selectedProducts, setSelectedProducts] = useState<Product[]>([]);
```

Рисунок 8 – Реализация работы хранения выбранных продуктов

- Управление списком избранных рецептов (favoriteRecipes);

```
const [favoriteRecipes, setFavoriteRecipes] = useState<Recipe[]>([]);
```

Рисунок 9 – Реализация работы управления списком избранных рецептов

- Фильтрация рецептов по совпадению ингредиентов;

```
const addToFavorites = (recipe: Recipe) => {  
  setFavoriteRecipes(prev => {  
    if (prev.some(r => r.title === recipe.title)) return prev;  
    return [...prev, recipe];  
  });  
};  
  
const removeFromFavorites = (recipeTitle: string) => {  
  setFavoriteRecipes(prev => prev.filter(recipe => recipe.title !== recipeTitle));  
};
```

Рисунок 10 – Реализация работы фильтрации

- Работа с localStorage для сохранения данных между сессиями.

```
const filteredRecipes = allRecipes
  .filter(recipe => {
    if (selectedCategory && recipe.category !== selectedCategory) {
      return false;
    }

    const selectedProductIds = selectedProducts.map(p => p.id);
    const matchingIngredients = recipe.requiredIngredients.filter(reqId =>
      selectedProductIds.includes(reqId)
    );

    return matchingIngredients.length > 0;
  })
  .map(recipe => {
    const selectedProductIds = selectedProducts.map(p => p.id);
    const matchingIngredients = recipe.requiredIngredients.filter(reqId =>
      selectedProductIds.includes(reqId)
    );

    return {
      ...recipe,
      matchCount: matchingIngredients.length,
      matchPercent: (matchingIngredients.length / recipe.requiredIngredients.length)
    };
  })
  .sort((a, b) => {
    return b.matchPercent - a.matchPercent;
  });
```

Рисунок 11 – Реализация работы с localStorage

Это позволило отказаться от передачи props через несколько уровней компонентов, улучшив производительность и гибкость.

App.tsx

Файл App.tsx определяет структуру маршрутов с использованием react-router-dom. Все страницы приложения доступны по следующим путям:

- / — главная;
- /categories — выбор категории;
- /meat, /vegetables, /animal-origin, /plant-origin — выбор ингредиентов;
- /recipe-list — список рецептов;

- /favorites — избранные рецепты;
- /about — информация о приложении.

```
const App: React.FC = () => {  
  return (  
    <Layout>  
      <Routes>  
        <Route path="/" element={<HomePage />} />  
        <Route path="/about" element={<AboutPage />} />  
        <Route path="/categories" element={<CategoriesPage />} />  
        <Route path="/meat" element={<MeatPage />} />  
        <Route path="/vegetables" element={<VegetablesPage />} />  
        <Route path="/animal-origin" element={<AnimalOriginPage />} />  
        <Route path="/plant-origin" element={<PlantOriginPage />} />  
        <Route path="/recipe-list" element={<RecipeListPage />} />  
        <Route path="/favorites" element={<FavoritesPage />} />  
      </Routes>  
    </Layout>  
  );  
};
```

Рисунок 12 – Реализация маршрутизации страниц

CategoryGrid

Компонент CategoryGrid предоставляет возможность выбрать одну из шести категорий блюд:

- Закуска;
- Суп;
- Напиток;
- Основное блюдо;
- Десерт;
- Гарнир;
- Выпечка.

```

const handleCategorySelect = (categoryId: string) => {
  if (selectedCategory === categoryId) {
    setSelectedCategory('');
  } else {
    setSelectedCategory(categoryId);
  }
};

return (
  <div className="categories__container">
    {categories.map((category, index) => (
      <div key={category.id} className={`category category--position-${index + 1}`}>
        <div
          className={`category__card ${selectedCategory === category.id ? 'category__card--selected' : ''}`}
          onClick={() => handleCategorySelect(category.id)}
        >
          <img src={category.imgSrc} alt={category.name} className="category__image" />
        </div>
        <h2 className="category__title">{category.name}</h2>
      </div>
    ))}
    <div className="nav-arrow nav-arrow--right" onClick={handleNextClick}>
      <div className="nav-arrow__symbol">>></div>
    </div>
  </div>
);

```

Рисунок 13 – Реализация выбора категории

Выбор категории влияет на фильтрацию рецептов. Реализована проверка на обязательный выбор категории перед переходом к следующему этапу.

ProductGrid

Универсальный компонент ProductGrid используется на нескольких страницах для выбора ингредиентов. Принимает массив продуктов, предыдущую и следующую страницы. Поддерживает множественный выбор.

Продукты разделены на группы:

- Мясо;
- Овощи;
- Животного происхождения;
- Растительного происхождения.

```

const toggleProduct = (product: Product) => {
  setSelectedProducts(prev => {
    const isSelected = prev.some(p => p.id === product.id);
    if (isSelected) {
      return prev.filter(p => p.id !== product.id);
    } else {
      return [...prev, product];
    }
  });
};

return (
  <div className="product-page__container">
    {products.map((product, index) => (
      <div key={product.id} className={`product product--position-${index + 1}`}>
        <div
          className={`product__card ${isSelected(product.id) ? 'product__card--selected' : ''}`}
          onClick={() => toggleProduct(product)}
        >
          <img src={product.imgSrc} alt={product.name} className="product__image" />
        </div>
        <h2 className="product__title">{product.name}</h2>
      </div>
    ))}
    <Link to={prevPage} className="nav-arrow nav-arrow--left">
      <div className="nav-arrow__symbol">⏪</div>
    </Link>
    <Link to={nextPage} className="nav-arrow nav-arrow--right">
      <div className="nav-arrow__symbol">⏩</div>
    </Link>
  </div>
);

```

Рисунок 14 – Реализация выбора продуктов

RecipeList

После того как пользователь выбрал категорию блюда и ингредиенты, выводится список подходящих рецептов. Они фильтруются по совпадению ингредиентов и категории, отсортированы по степени совпадения (matchPercent).

Каждый рецепт содержит:

- Название;
- Ингредиенты;
- Возможность добавления в избранное.

```

<div className="recipe-list">
  <h2>Рецепты:</h2>
  <ul id="recipe-list-ul">
    {filteredRecipes.length > 0 ? (
      filteredRecipes.map(recipe => (
        <li
          key={recipe.title}
          onClick={() => handleRecipeClick(recipe)}
        >
          <strong>{recipe.title}</strong>
          <small className="recipe-ingredients">
            Ингредиенты: {recipe.requiredIngredients.map(ing => (
              ingredientTranslations[ing as keyof typeof ingredientTranslations] || ing
            )).join(', ')}
          </small>
          <span
            className={`favorite-icon ${isInFavorites(recipe.title) ? 'favorited' : ''}`}
            onClick={(e) => toggleFavorite(e, recipe)}
          >
            {isInFavorites(recipe.title) ? '♥' : '♡'}
          </span>
        </li>
      ))
    ) : (
      <li>Подходящих рецептов не найдено</li>
    )}
  </ul>
</div>
</div>

```

Рисунок 15 – Реализация вывода рецептов

RecipeModal

Модальное окно RecipeModal позволяет просматривать полное описание рецепта. Реализована возможность добавления/удаления из избранного прямо из окна.

```

return (
  <div className="modal" onClick={onClose}>
    <div className="modal-content" onClick={e => e.stopPropagation()}>
      <span className="modal-close-button" onClick={onClose}>&times;</span>
      <h2 id="modal-recipe-title">{recipe.title}</h2>
      <span
        className={`favorite-icon ${isFavorite ? 'favorited' : ''}`}
        onClick={toggleFavorite}
      >
        {isFavorite ? '♥' : '♡'}
      </span>
      <div
        id="modal-recipe-content"
        dangerouslySetInnerHTML={{ __html: formattedRecipe }}
      />
    </div>
  </div>
);

```

Рисунок 16 – Реализация просмотра полного рецепта

3.3 Заключение по главе

В ходе реализации проекта было разработано полноценное веб-приложение для поиска рецептов. При создании приложения были использованы современные веб-технологии, позволившие обеспечить высокую производительность и удобство использования. Были реализованы ключевые компоненты: интерфейс пользователя, система поиска рецептов по ингредиентам, модуль управления состоянием приложения, а также адаптация под различные устройства. Также была настроена логика сохранения данных и взаимодействия с серверной частью, что позволило создать устойчивую и масштабируемую архитектуру. Таким образом, этап реализации завершён успешно, и результат может быть использован как основа для дальнейшего развития.

Глава 4. Тестирование

Тестирование проводилось по следующим направлениям:

- Проверка корректной работы выбора категории;
- Отслеживание поведения при нажатии «Назад» и «Вперед»;
- Проверка системы выбора ингредиентов;
- Тестирование фильтрации рецептов по категории и наличию ингредиентов;
- Проверка добавления/удаления из избранного;
- Проверка работы модального окна;
- Тестирование работы с хранилищем localStorage.

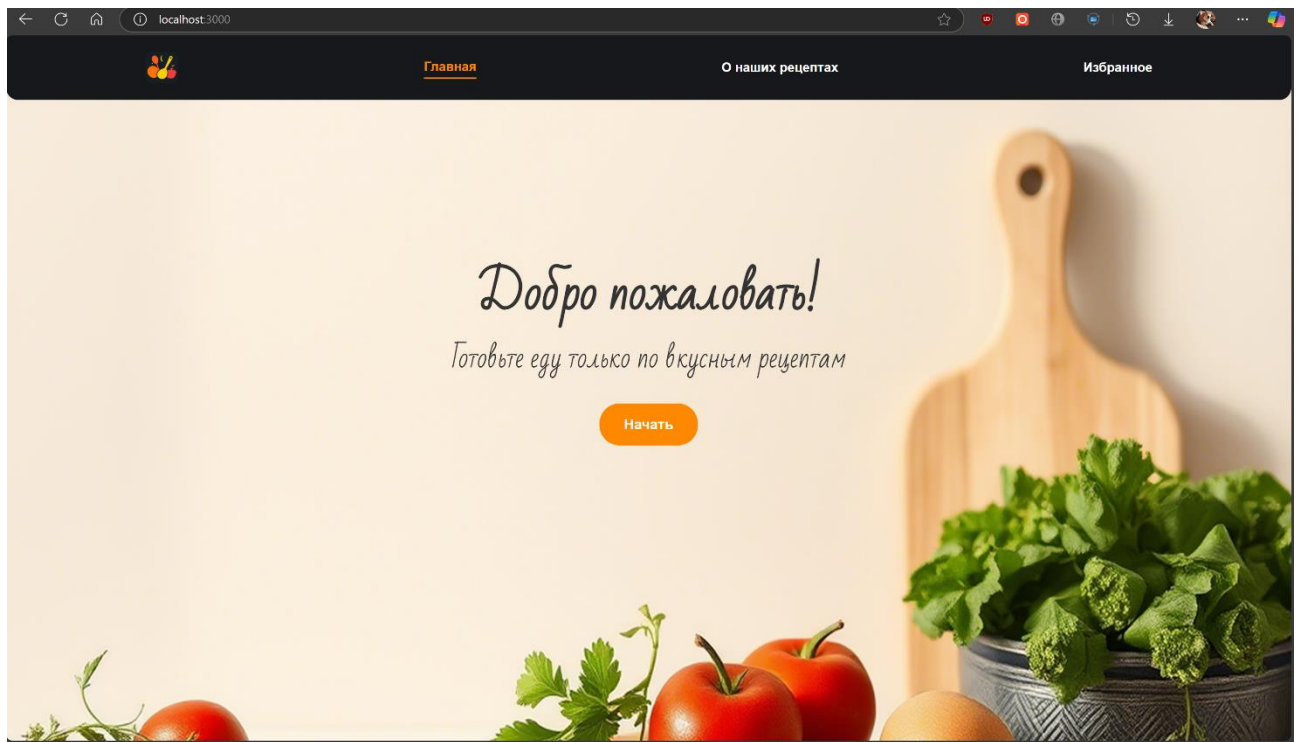


Рисунок 17 – Страница входа

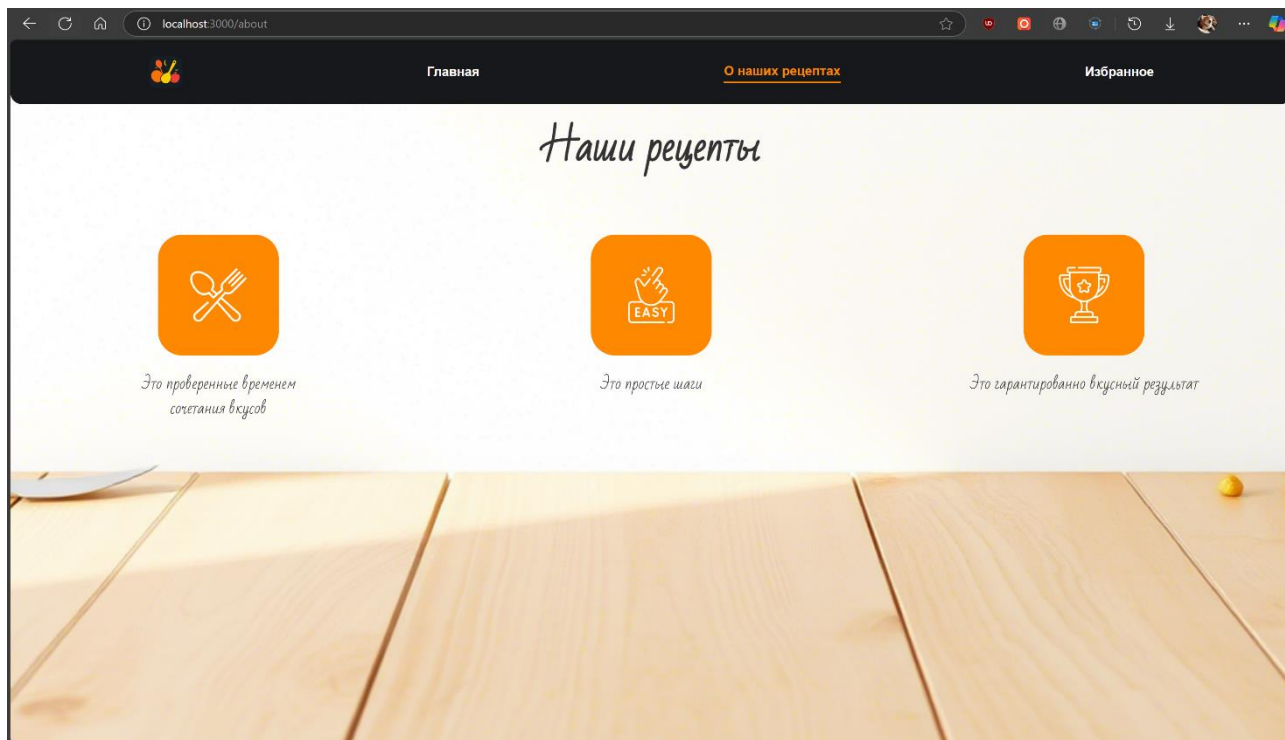


Рисунок 18 – Страница «О нас»

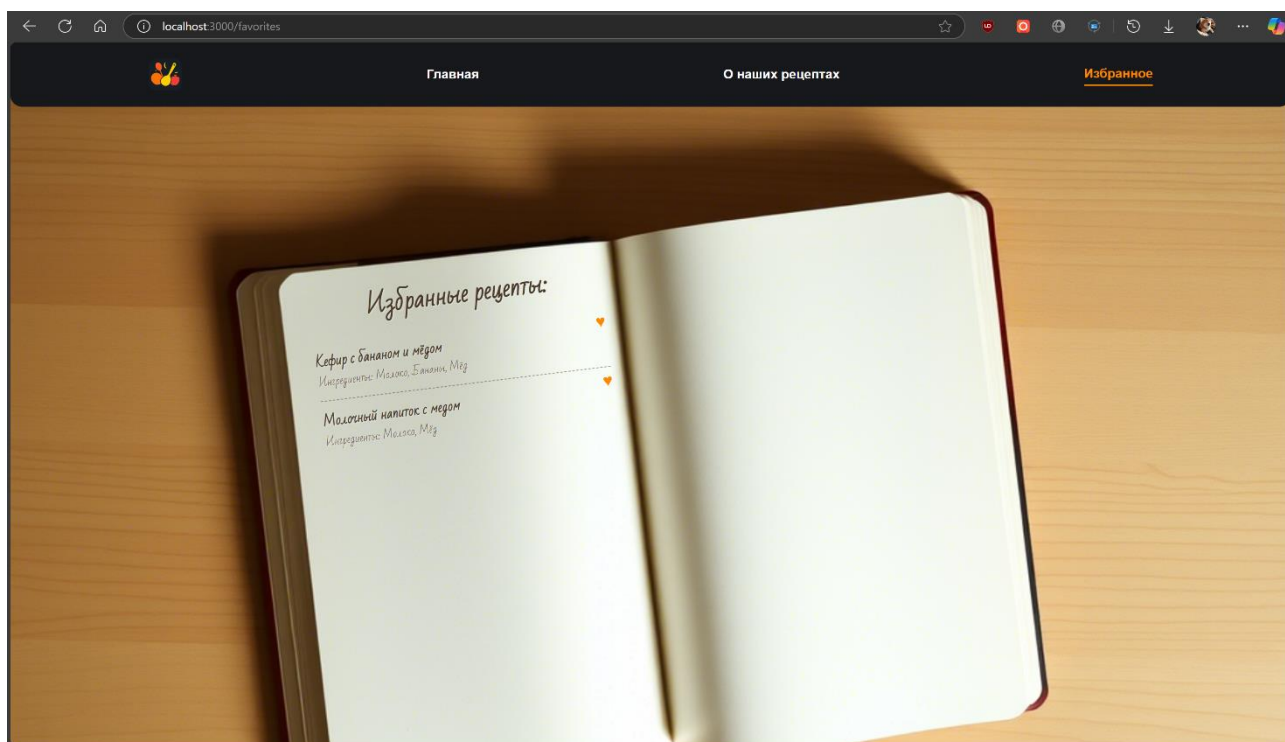


Рисунок 19 – Избранные рецепты

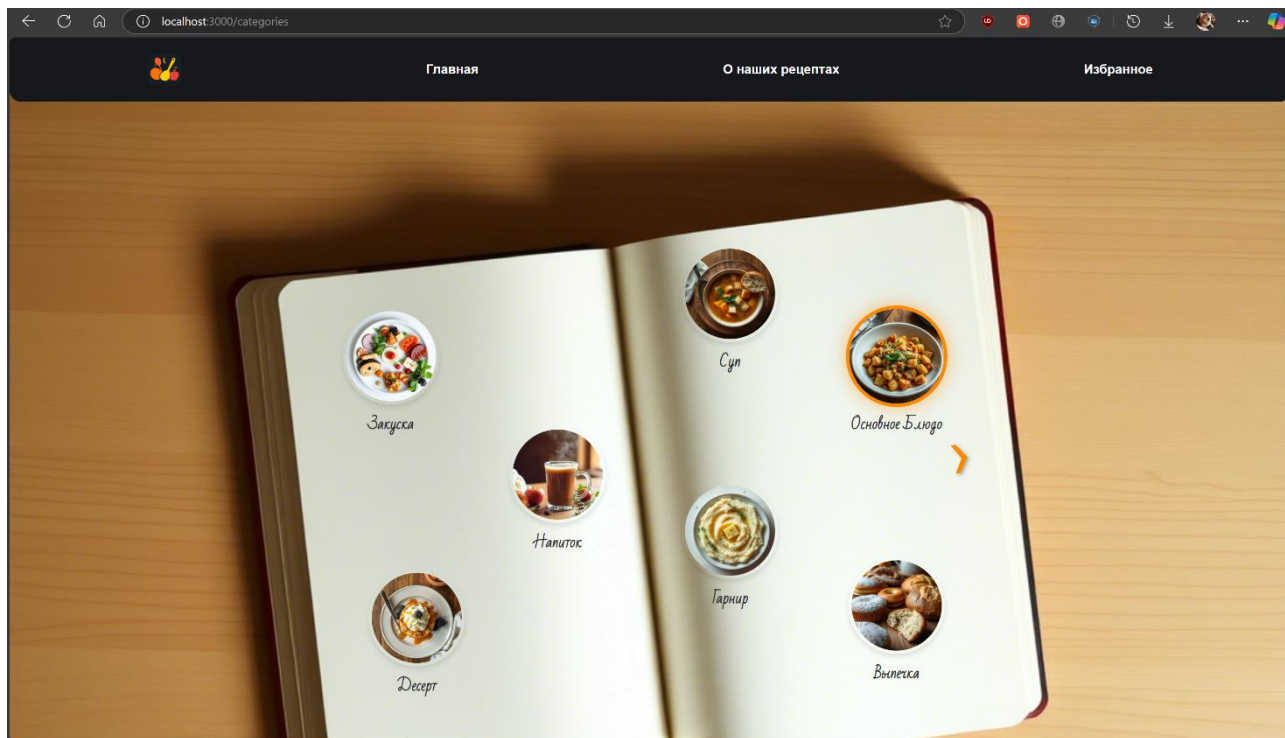


Рисунок 20 – Выбор категории блюда

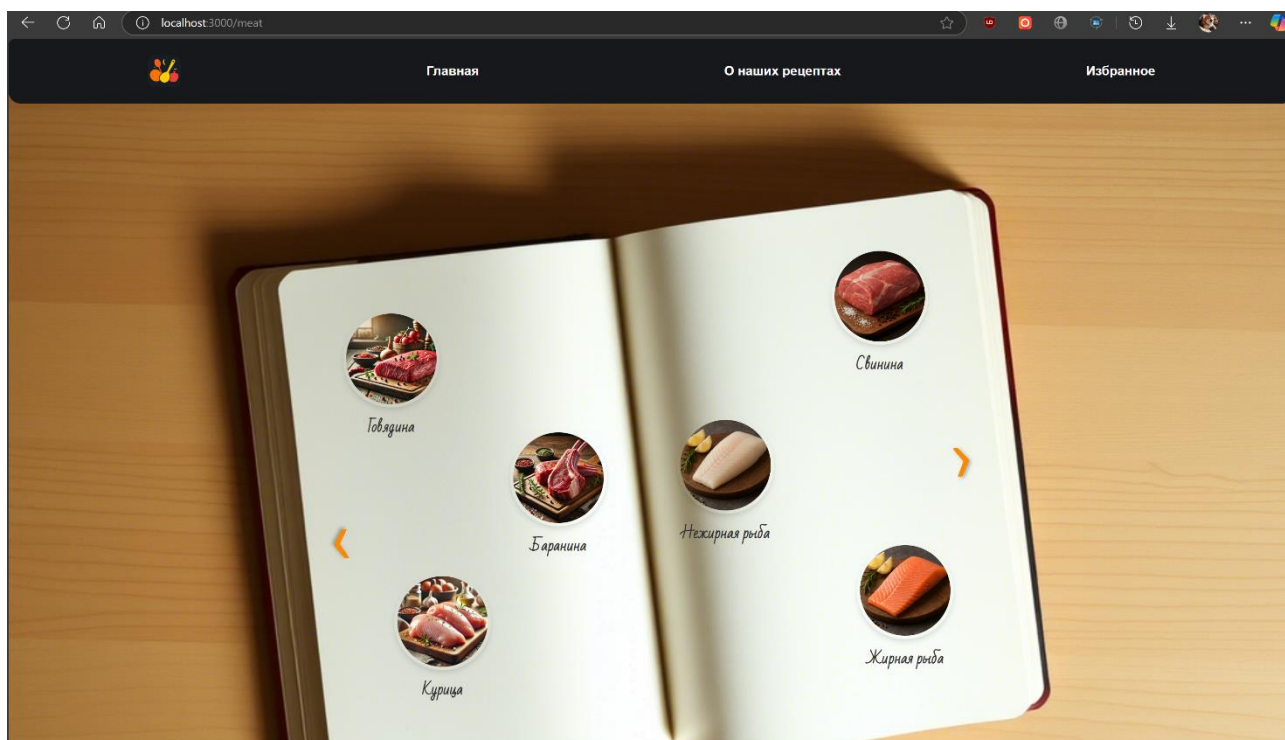


Рисунок 21 – Выбор мяса и рыбы

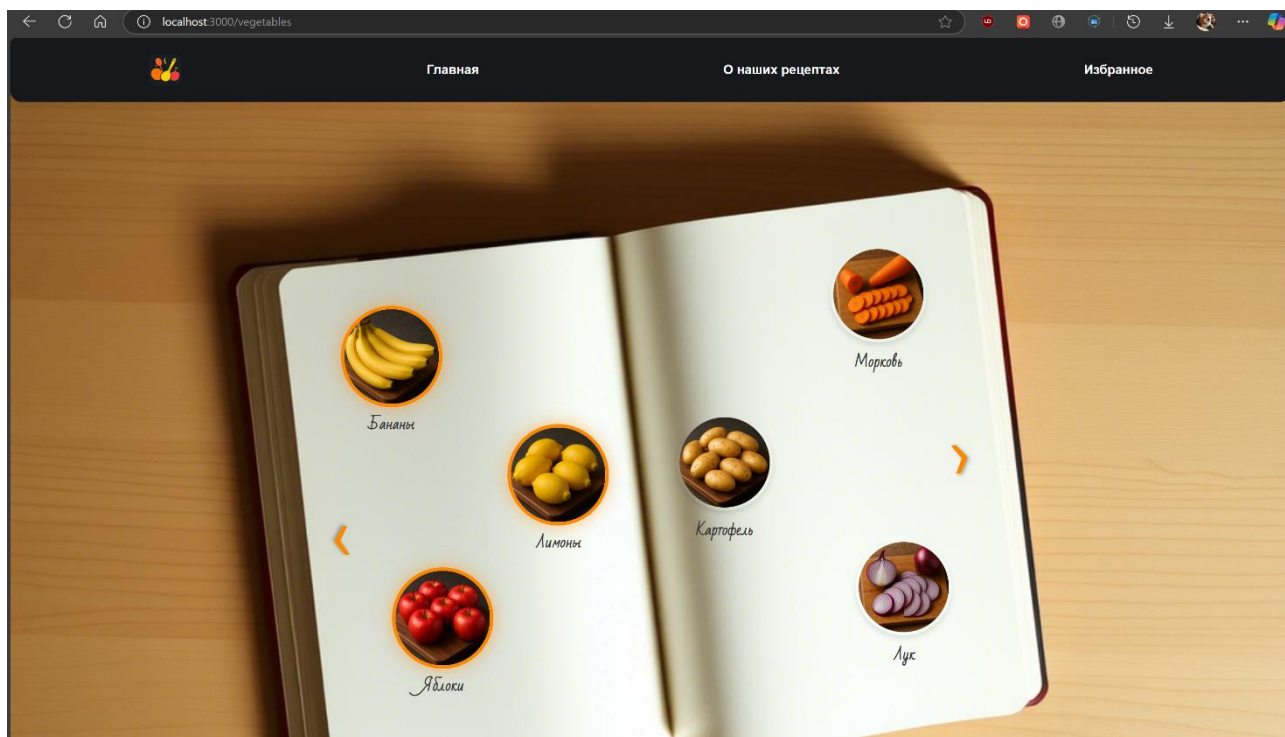


Рисунок 22 – Выбор овощей и фруктов

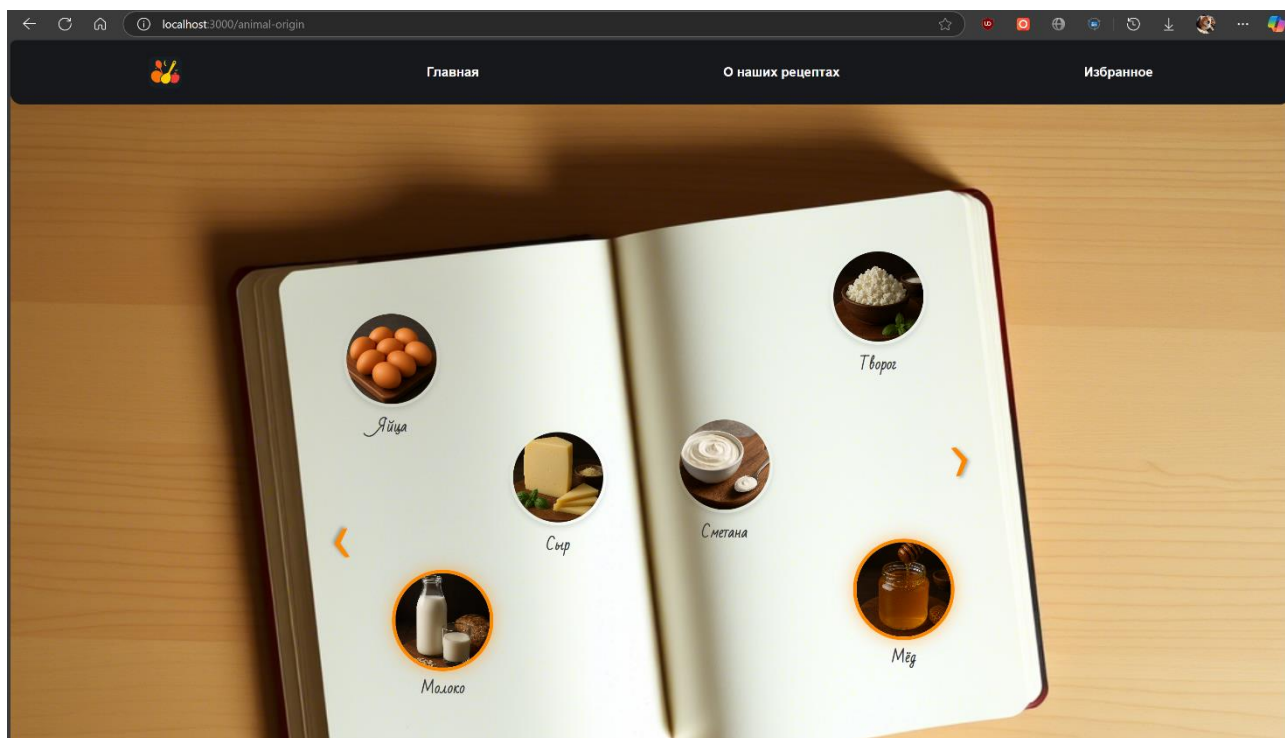


Рисунок 23 – Выбор продуктов животного происхождения

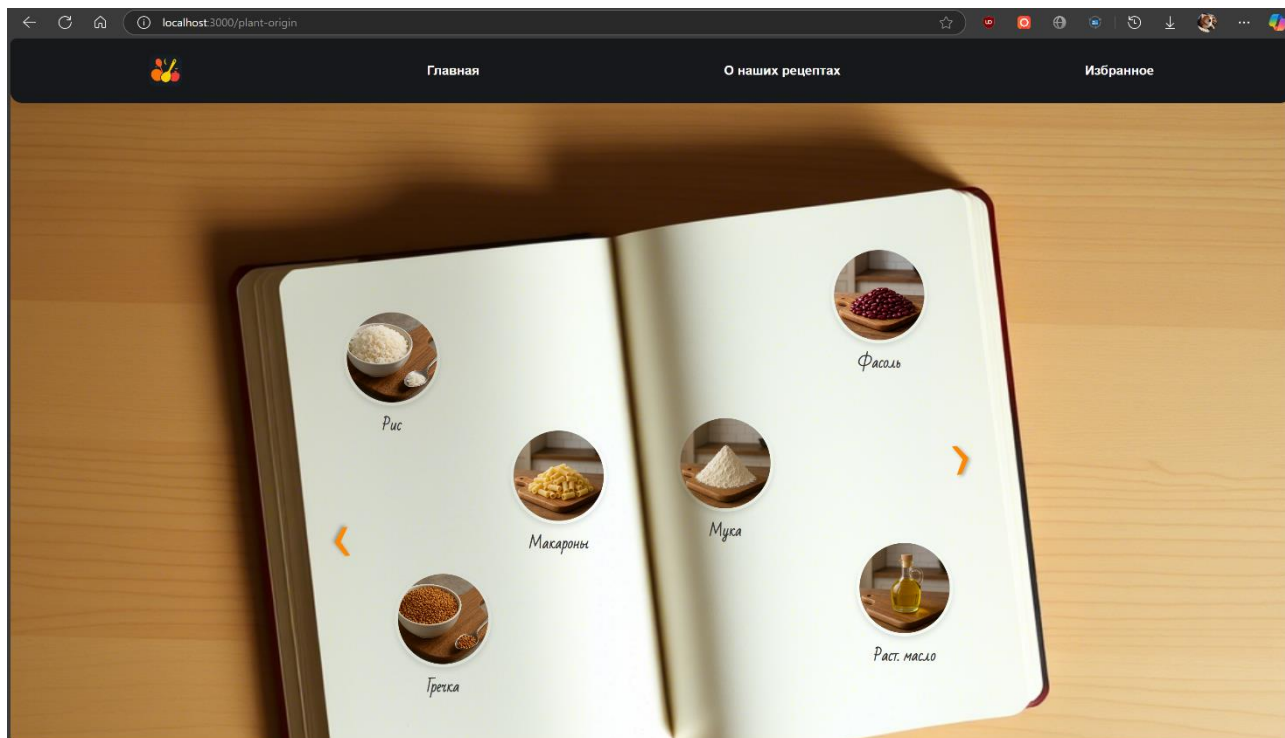


Рисунок 24 – Выбор продуктов растительного происхождения

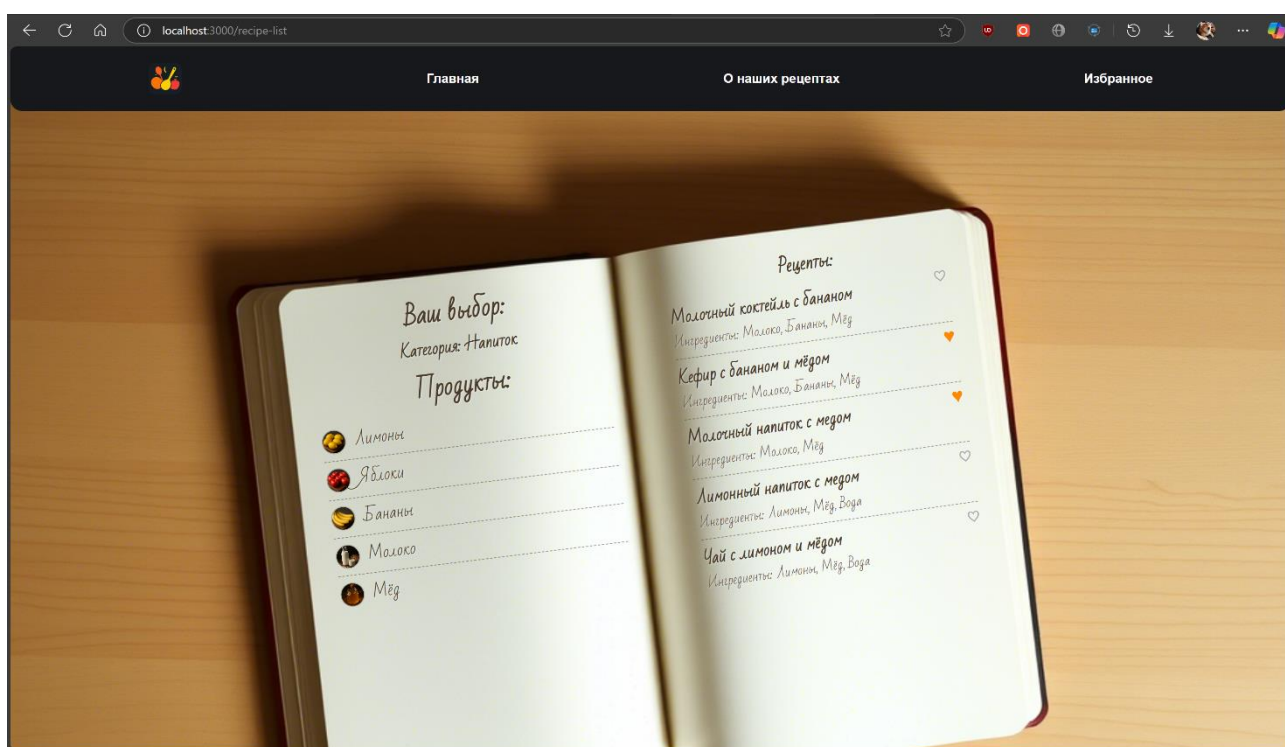
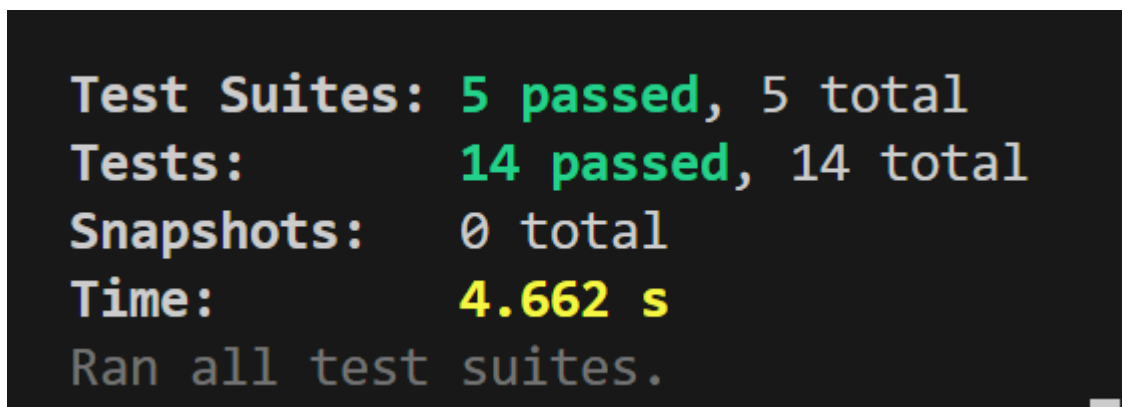


Рисунок 25 – Итоговая страница, вывод подходящих рецептов

В проекте реализована система модульного тестирования с использованием Jest, охватывающая ключевые компоненты фронтенд-части приложения. Тестирование начинается с проверки базовых элементов интерфейса, таких как навигационная панель в компоненте Layout, где

тестируется отображение ссылок «Главная», «О наших рецептах», «Избранное», а также корректное отображение дочернего контента и логотипа. Компонент CategoryGrid проверяется на корректность выбора категории блюда: тесты убеждаются, что без выбора категории выводится предупреждение, а при успешном выборе происходит переход на следующий шаг маршрутизации. В ProductGrid тестируется отрисовка продуктов, возможность их выбора и переключения состояния, а также наличие навигационных кнопок. Компонент RecipeList проверяет отображение выбранных категории и ингредиентов, вывод списка рецептов и сообщения об отсутствии подходящих вариантов. Модальное окно RecipeModal тестируется на корректное отображение деталей рецепта, работу кнопки закрытия и условие отрисовки при изменении состояния isOpen. Все тесты выполняются с использованием RecipeProvider, который обеспечивает контекстное управление состоянием, а также используются моки для react-router-dom и других зависимостей для изоляции компонентов. Конфигурация Jest позволяет корректно работать с React-компонентами и контекстом, обеспечивая надёжное и стабильное тестирование. Результаты выполнения тестов представлены на рисунке 26.



```
Test Suites: 5 passed, 5 total
Tests:      14 passed, 14 total
Snapshots:  0 total
Time:       4.662 s
Ran all test suites.
```

Рисунок 26 – Jest тестирование

В проекте реализованы комплексные E2E-тесты с использованием Cypress, которые проверяют функциональность приложения для подбора рецептов в ключевых пользовательских сценариях. Все тесты сосредоточены в отдельных

файлах, соответствующих функциональным модулям: навигация, выбор рецептов, работа с избранным и модальное окно просмотра рецепта.

Тесты охватывают такие сценарии, как переход по основным страницам приложения (главная, «О наших рецептах», «Избранное»), проверка корректности маршрутов и отображения контента. Также протестировано начало пользовательского сценария подбора рецептов — выбор категории блюда, ингредиентов и вывод подходящих рецептов. Проверено поведение системы при попытке перехода без выбора категории — отображение предупреждения. В модальном окне рецепта тестируется открытие, закрытие и возможность добавления/удаления рецепта из избранного прямо внутри модального окна.

Функционал работы с избранными рецептами также полностью проверен: начиная с добавления первого рецепта в избранное и заканчивая проверкой его наличия на странице «Избранное» и отсутствием сообщения об отсутствии избранных рецептов. Все тесты имитируют реальные действия пользователя — клики, выбор элементов, переходы между страницами, что делает их максимально приближенными к реальному использованию приложения.

Для устойчивости тестов используются точные селекторы и проверки состояния DOM-элементов, а также гарантируется чистое состояние приложения перед каждым тестом. Результат прогона Cypress-тестов представлен на рисунках 27–30, включая тестирование избранного, навигации, модального окна и подбора рецептов.

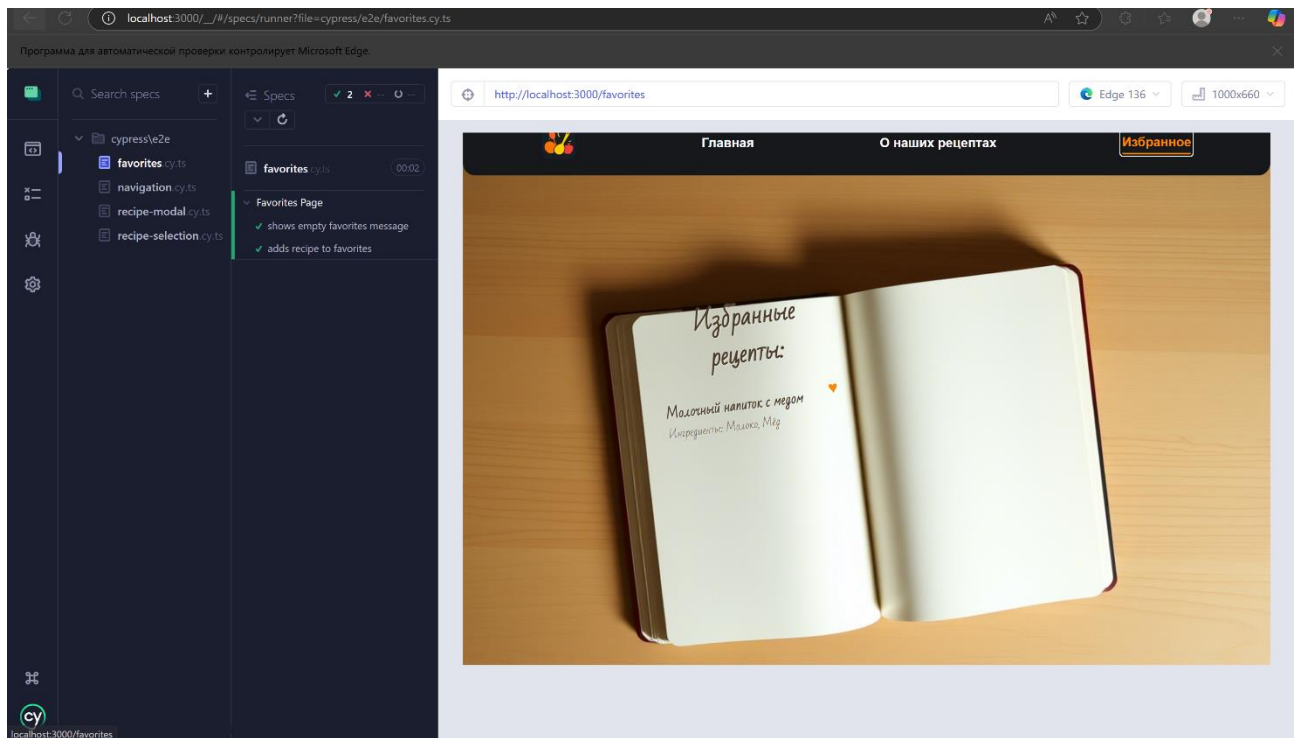


Рисунок 27 – Cypress тестирование избранных рецептов

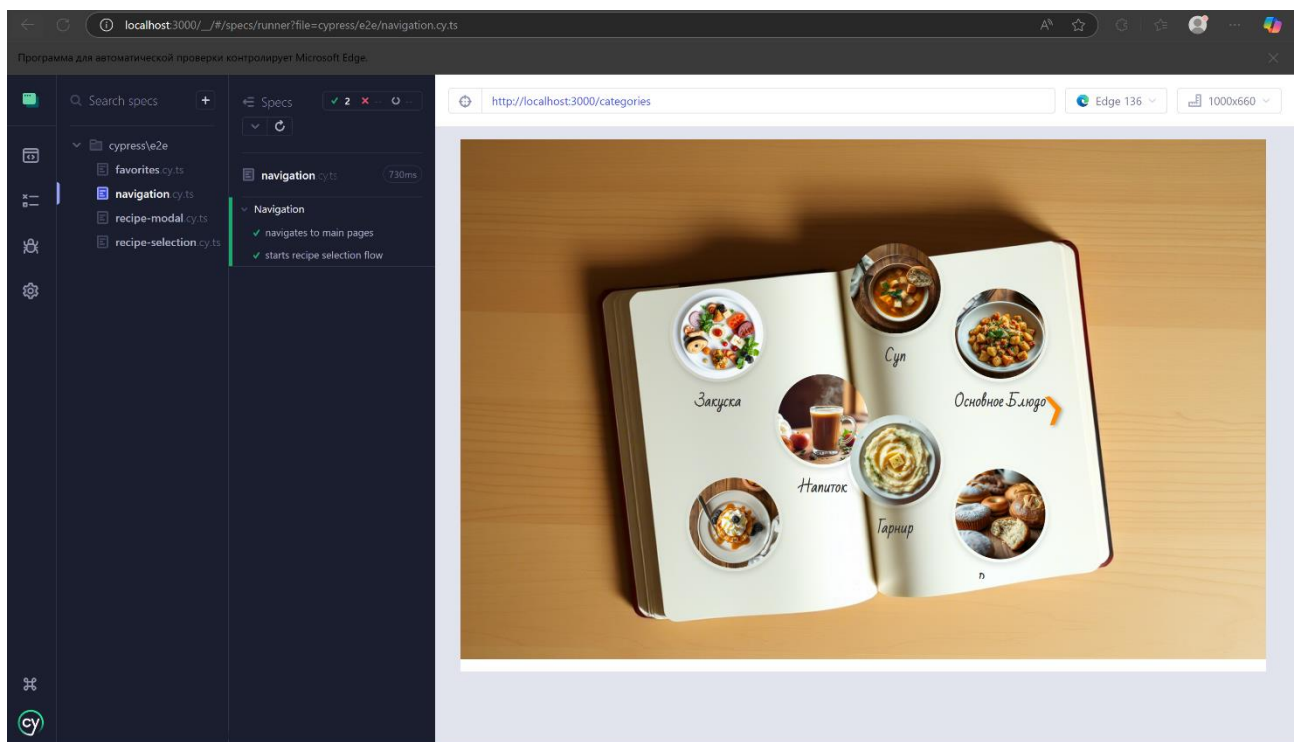


Рисунок 28 – Cypress тестирование навигации

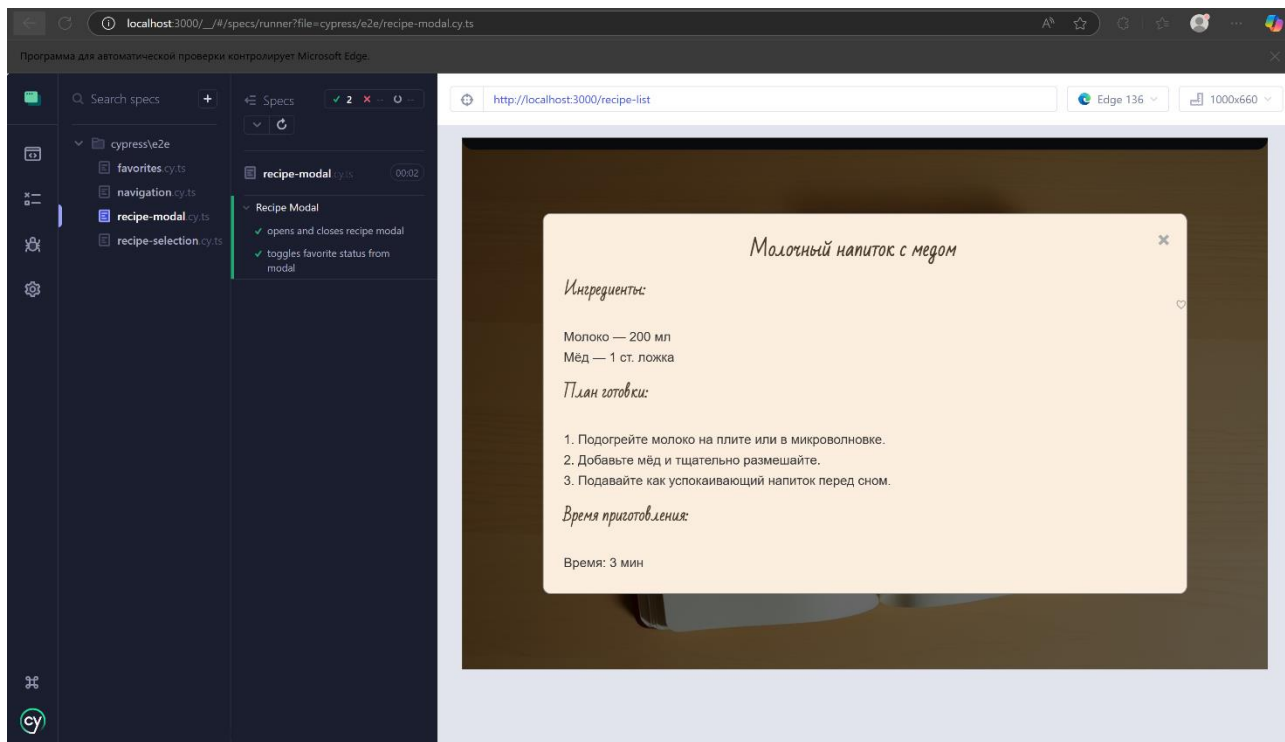


Рисунок 29 – Cypress тестирование модального окна

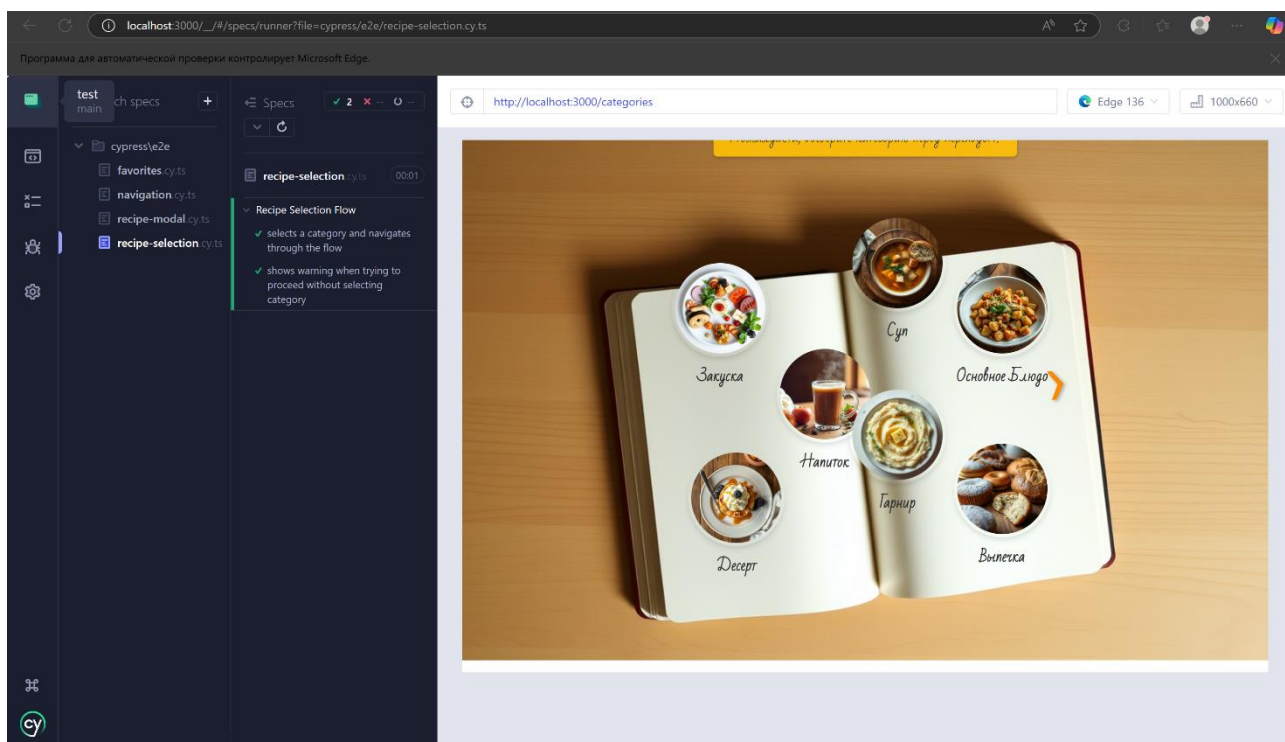


Рисунок 30 – Cypress тестирование подбор рецепта

На этапе тестирования проведена полная проверка работоспособности веб-приложения. Проверено соответствие реализованных функций изначальным требованиям. Все ключевые модули приложения работают корректно, ошибки,

выявленные на ранних стадиях, успешно исправлены. Протестирована совместимость приложения на различных устройствах и браузерах, реализована адаптивная верстка. Управление состоянием и работа с данными протестированы на стабильность и надёжность. В результате тестирования подтверждена готовность приложения к демонстрации и возможному внедрению в качестве базового решения сервиса по поиску рецептов.

Заключение

В ходе выполнения курсовой работы была достигнута поставленная цель — разработано веб-приложение, позволяющее пользователю выбирать категорию блюда и указывать доступные ингредиенты, после чего система предлагает список подходящих рецептов. Приложение реализовано с использованием современных веб-технологий: React + TypeScript, что обеспечило модульность, типизацию и удобство поддержки кода.

Все задачи, поставленные на начальном этапе, успешно выполнены:

- Проведен анализ предметной области и изучены конкурентные решения;
- Сформулированы функциональные и нефункциональные требования к продукту;
- Разработана концепция приложения, включающая логику работы, навигационную структуру и архитектуру данных;
- Выбраны и обоснованы технологии реализации;
- Создан дизайн интерфейса, ориентированный на удобство использования;
- Реализовано клиентское приложение с полной функциональностью;
- Проведено тестирование всех компонентов и исправлены выявленные ошибки;

Приложение демонстрирует высокую степень юзабилити и адаптируется под различные устройства. Функционал включает выбор категории блюда, выбор ингредиентов, фильтрацию рецептов, просмотр подробного описания и добавление в избранное. Вся информация хранится локально (через localStorage), что позволяет сохранять предыдущие действия пользователя между сессиями без необходимости регистрации или серверной части.

Проект может быть легко расширен: возможно добавить систему авторизации, интеграцию с API внешних сервисов, базу данных,

дополнительные фильтры (например, по времени приготовления или диетическим ограничениям) и улучшенный алгоритм подбора рецептов.

Таким образом, результатом курсовой работы стало полностью работоспособное веб-приложение, соответствующее поставленным целям и требованиям. Оно может служить как учебным проектом, так и основой для полноценного сервиса по поиску рецептов.

Список использованных источников

1. Дюнаев С. В. React и TypeScript: разработка масштабируемых приложений. — М.: Диалектика, 2021. — 328 с.
2. Бэнкс А., Порселло Е. *Современный React. Полное руководство*. — СПб.: Питер, 2022. — 496 с.
3. Документация React — <https://reactjs.org/docs/getting-started.html> (дата обращения: 10.05.2025).
4. Документация TypeScript — <https://www.typescriptlang.org/docs/> (дата обращения: 10.05.2025).
5. React Router Documentation — <https://reactrouter.com/en/main/start/tutorial> (дата обращения: 10.05.2025).
6. Mozilla Developer Network (MDN). JavaScript Guide — <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide> (дата обращения: 09.05.2025).
7. Стандарт ГОСТ Р 7.0.5-2008. Библиографическая ссылка. Общие требования и правила составления. — М.: Стандартинформ, 2008.
8. Официальный сайт Figma — <https://www.figma.com> (дата обращения: 11.05.2025).
9. Советы по UX-дизайну интерфейсов — NNGroup, Jakob Nielsen, <https://www.nngroup.com> (дата обращения: 12.05.2025).
10. Трофимов И. В. Разработка пользовательских интерфейсов с использованием React и Redux. — М.: БХВ-Петербург, 2020. — 384 с.